SOFTWARE ARCHITECTURE AND DESIGN

Student Portfolio Showcase

SAD • Proposed architecture and component design

1. Introduction

This document translates the SRS into a proposed architecture, component structure, data model, interfaces, security controls and CI/CD design. It is a design baseline, not proof that every proposed feature exists in the current code.

1.1 Design principles

- Use a modular Flask application appropriate for a student mini-project.
- Separate routes, business rules, persistence and presentation.
- Keep database access parameterised and testable.
- Enforce authentication and ownership on the server.
- Automate repeatable quality checks through GitHub Actions.
- Prefer readable, testable code over unnecessary infrastructure.

2. Architecture overview

Proposed layered web architecture: Browser → Flask routes/controllers → application services → data-access layer → SQLite. HTML templates/static assets render the interface. pytest validates behaviour; Ruff and GitHub Actions check repository changes.

Layer	Responsibility	Proposed elements
Presentation	Render pages and collect input	HTML templates, CSS, JavaScript
Web/API	HTTP routing, sessions and responses	Flask app factory, blueprints/routes
Application	Use cases, validation and business rules	AuthService, PortfolioService, validators
Persistence	Read/write records	SQLAlchemy OR parameterised SQLite repository; choose one
Data	Persistent records	SQLite
Quality/delivery	Automated verification and integration	pytest, Ruff, GitHub Actions, PR review

2.1 Deployment view

For development/evaluation, run locally with Python 3.11 and SQLite. If public deployment is required, use an instructor-approved host, environment-based secrets, HTTPS and persistent database storage. Hosting provider remains unconfirmed.

3. Component design

Component	Responsibility	Suggested location
Application factory	Create/configure Flask app and register modules	app/__init__.py
Authentication	Registration, login, logout, hashing and session checks if in scope	app/auth/
Portfolio	Manage profile, skills, projects and achievements	app/portfolio/

Public portfolio	Resolve public slug and expose published fields only	app/public/
Validation/errors	Validate fields and map errors to safe responses	app/validators.py; app/errors.py
Data layer	Models, relationships and database operations	app/models.py; app/extensions.py
Presentation	Pages, forms, public view and styling	app/templates/; app/static/
Tests	Unit, route, integration and security-oriented tests	tests/
CI workflow	Install, lint and test pull requests/pushes	.github/workflows/ci.yml

4. Data design

Proposed relational model. Final field definitions and relationships must be aligned with implemented features before database schema is treated as final.

Entity	Key fields	Relationships / constraints
User	id PK, email unique, password_hash, created_at	One user owns a portfolio; never store plaintext password.
Portfolio	id PK, user_id FK, slug unique, display_name, headline, bio, is_published, updated_at	Belongs to User; public route returns only if published.
Skill	id PK, portfolio_id FK, name, category, proficiency	Many skills per portfolio.
Project	id PK, portfolio_id FK, title, description, technologies, repo_url, demo_url	Many projects per portfolio.
Achievement	id PK, portfolio_id FK, title, issuer, awarded_on, credential_url	Many achievements per portfolio.

4.1 Integrity rules

- Use primary/foreign keys and unique email/slug constraints.
- Validate URLs and required fields before persistence.
- Use cascading deletion only when explicitly intended and tested.
- Use isolated test database or temporary database for automated tests.

5. Interface and API design

The endpoints below are proposals and may be renamed to match the final implementation. All modifying routes require authentication and resource ownership checks.

Method	Proposed endpoint	Purpose	Access
GET	/	Landing/status	Public
GET	/health	Health status	Public
POST	/auth/register	Create account	Public, if enabled
POST	/auth/login	Authenticate owner	Public, if enabled
POST	/auth/logout	End session	Authenticated
GET/POST	/portfolio	View/create portfolio	Authenticated
GET/PUT	/portfolio/profile	View/update profile	Authenticated
POST/PUT/DELETE	/portfolio/skills/:id	Manage skills	Owner only
POST/PUT/DELETE	/portfolio/projects/:id	Manage projects	Owner only
POST/PUT/DELETE	/portfolio/achievements/:id	Manage achievements	Owner only
POST	/portfolio/publish	Publish/unpublish	Owner only
GET	/p/<slug>	Public portfolio	Published only

5.1 Response conventions

- Use suitable status codes: 200/201 success, 400 invalid input, 401 unauthenticated, 403 forbidden, 404 missing resource.
- Use consistent JSON errors for API routes and clear messages for HTML forms.
- Do not return stack traces, secrets or sensitive account details.

- Use parameterised SQL or a maintained ORM.

6. Security design

Concern	Control	Verification
Credential disclosure	Salted password hashes; never log passwords	Auth tests/code review
Cross-user access	Check ownership on private reads/writes	Attempt cross-user access
Session misuse	Secure session configuration; invalidate on logout	Inspect config and test logout
Injection/input	Validate fields; parameterised SQL/ORM	Negative tests/code review
Secret leakage	Environment variables/hosting secret store	Repository scan/review
Information leakage	Safe errors and minimal logs	Inspect error responses/logs
Network interception	HTTPS for public deployment	Verify TLS/redirect

7. Quality attributes and trade-offs

Attribute	Design response	Trade-off / note
Maintainability	Modular Flask structure and separate tests	More files than a single-file prototype
Testability	App factory, test config and isolated DB	Requires test setup discipline
Portability	Documented dependencies and setup	Deployment config remains environment-specific
Performance	SQLite for small demo scale	Not intended for high concurrent production load
Security	Server-side validation and ownership checks	Must be implemented and tested
Usability	Consistent navigation, labels and feedback	Requires manual review

8. Repository and CI/CD design

The shared GitHub repository is the single source of truth. Contributors work on branches, open pull requests, request review and merge after required checks pass.

Path	Purpose
app/	Flask package, routes, models, services and templates
tests/	pytest unit/integration/regression tests
.github/workflows/ci.yml	GitHub Actions lint/test workflow
requirements.txt	Project dependencies
pyproject.toml	pytest/Ruff configuration
run.py	Local development entry point
README.md	Setup, configuration, test and run instructions

8.1 CI pipeline stages

1. Checkout repository source.
2. Set up agreed Python version.
3. Install dependencies.
4. Run Ruff lint checks.
5. Run pytest; fail workflow on test failure.
6. Show pass/fail status on pull request; configure branch protection to require checks where available.

9. Risks and mitigations

Risk	Impact	Mitigation
Scope too large	Incomplete core product	Prioritise profile/project/public view and CI; defer extras with approval.
Authentication unclear	Rework	Confirm scope before implementing auth.

Integration conflicts	Broken main branch	Branches, PR review, small merges and CI.
Late schema changes	Data/test failures	Agree model early; use controlled schema changes.
Hosting unavailable	CD demo blocked	Confirm provider early; retain local run as fallback if accepted.

10. Decisions requiring confirmation

The following technical decisions have been established for the Student Portfolio Showcase project to ensure consistency during development, integration, testing, and deployment.

1. Database and Persistence Approach

- SQLite will be used as the relational database for storing user accounts, portfolio details, projects, and skills.
- SQLAlchemy will be used as the Object Relational Mapper (ORM) to manage database operations.
- Database models will be maintained separately to ensure modularity and maintainability.

2. User Registration and Authentication

- The application will support user registration and login.
- Registered users will be able to create, update, and manage their own portfolios.
- Passwords will be securely hashed before being stored in the database.
- Authentication and access control will be implemented to prevent unauthorised access to user-specific information.

3. Public Deployment

- The Student Portfolio Showcase will be deployed publicly so that users can access the application through a web browser.
- The application will use a publicly accessible hosting platform that supports Python Flask applications.
- Environment variables will be used to manage sensitive configuration values.
- The deployment process will be integrated with GitHub Actions to support automated testing and continuous integration.
- The final public deployment URL will be documented in the project README.

4. Portfolio Content and File Handling

- Users will be able to add and manage personal information, skills, project descriptions, and project links.
- External links to GitHub repositories and other project resources will be supported.
- File and image uploads may be added if required by the final implementation and supported by the deployment environment.

5. Application Architecture and Module Organisation

- The application will follow a modular Flask architecture.
- Backend routes, database models, authentication logic, and frontend components will be organised into separate modules wherever appropriate.
- Actual module names, API endpoints, and implementation details will be documented after integration.

6. Architecture Diagrams and Documentation

- Architecture diagrams, database diagrams, and component descriptions will be maintained as part of the Software Architecture and Design documentation.
- Documentation will be updated to reflect the final implemented system.
- Any deviations from the proposed architecture will be recorded and explained during the final evaluation.